

Comece a programar ou gere código com IA.

15. Exercício 1 — Referências Executar:

```
a = [10, 20, 30] b = a
```

```
b.append(40)
```

```
print("a:", a) print("b:", b) print("id(a):", id(a)) print("id(b):", id(b))
```

 Responde:

Qual será a saída?

```
a: [10, 20, 30, 40]
b: [10, 20, 30, 40]
id(a): 140123456789456
id(b): 140123456789456
```

2. a e b representam o mesmo objeto?

```
a = [10, 20, 30]
b = a
```

```
print(a is b)
# Saída: True
```

3. Por quê?

```
# Porque b = a faz as duas variáveis apontarem para a mesma lista.
```

```
b.append(40)
```

```
print(a)
print(b)
# Saída:
# [10, 20, 30, 40]
# [10, 20, 30, 40]
```

4. O que aconteceria com b = a.copy()?

```
a = [10, 20, 30]
b = a.copy()
```

```
b.append(40)
```

```
print(a)
print(b)
print(a is b)
```

16. Exercício 2 — Construindo uma cadeia Chorar:

```
n1 = Node("A") n2 = Node("B") n3 = Node("C")
```

 Conectar:

A → B → C → None Depois percorra a estrutura e produza:

A B C

```
class Node:
    def __init__(self, valor):
        self.valor = valor
        self.proximo = None
```

```
# Criando os nós
n1 = Node("A")
n2 = Node("B")
n3 = Node("C")
```

```
# Conectando os nós
n1.proximo = n2
n2.proximo = n3
n3.proximo = None
```

```
# Percorrendo a estrutura
atual = n1

while atual is not None:
    print(atual.valor)
    atual = atual.proximo
```

17. Exercício 3 — Depuração Analisar:

```
n1 = Node(10) n2 = Node(20) n3 = Node(30)
```

```
n1.proximo = n2 n2.proximo = n3
```

```
atual = n1
```

```
while atual is not None: print(atual.valor) Perguntas:
```

Qual é o problema?

```
O problema é que atual nunca é atualizado dentro do while. Ele fica sempre apontando para n1, então o programa imprime 10 :

n1 = Node(10)
n2 = Node(20)
n3 = Node(30)

n1.proximo = n2
n2.proximo = n3

atual = n1

while atual is not None:
    print(atual.valor)
    atual = atual.proximo
```

2. Por que o programa não termina?

o programa não termina porque atual permanece apontando para o mesmo nó (n1) e o while nunca chega a None

3. Qual linha deve ser acrescentada?

```
atual = atual.proximo
```

4. Qual será a saída depois da correção?

```
# Saída depois da correção:

10
20
30
```

18. Problema real — Sistema de atendimento de uma clínica Uma clínica recebe pacientes durante o dia. Cada paciente possui:

nome; idade; prioridade. Exemplo:

Ana — 32 anos — Normal Bruno — 70 anos — Prioridade Carlos — 45 anos — Normal O sistema deve permitir:

adicionar paciente; listar pacientes; atender o primeiro paciente; verifique se a fila está vazia; informar a quantidade de pacientes.

```
class Paciente:
    def __init__(self, nome, idade, prioridade):
        self.nome = nome
        self.idade = idade
        self.prioridade = prioridade

class Fila:
    def __init__(self):
        self.pacientes = []

    # 1. Adicionar paciente
    def adicionar(self, paciente):
        self.pacientes.append(paciente)

    # 2. Listar pacientes
    def listar(self):
        for paciente in self.pacientes:
```

```

        print(
            paciente.nome,
            "-",
            paciente.idade,
            "anos -",
            paciente.prioridade
        )

# 3. Atender o primeiro paciente
def atender(self):
    if not self.vazia():
        return self.pacientes.pop(0)
    return None

# 4. Verificar se a fila está vazia
def vazia(self):
    return len(self.pacientes) == 0

# 5. Informar a quantidade de pacientes
def quantidade(self):
    return len(self.pacientes)

# Criando a fila
fila = Fila()

# Adicionando pacientes
fila.adicionar(Paciente("Ana", 32, "Normal"))
fila.adicionar(Paciente("Bruno", 70, "Prioridade"))
fila.adicionar(Paciente("Carlos", 45, "Normal"))

# Listar pacientes
print("Pacientes na fila:")
fila.listar()

# Quantidade de pacientes
print("\nQuantidade:", fila.quantidade())

# Verificar se está vazia
print("Fila vazia?", fila.vazia())

# Atender o primeiro paciente
paciente = fila.atender()

print("\nPaciente atendido:")
print(paciente.nome, "-", paciente.idade, "anos -", paciente.prioridade)

# Quantidade depois do atendimento
print("\nQuantidade:", fila.quantidade())

```

19. Modelagem Chorar:

class Paciente: ... Depois:

class Node: ... O nó deverá:

paciente próximo

```

class Paciente:
    def __init__(self, nome, idade, prioridade):
        self.nome = nome
        self.idade = idade
        self.prioridade = prioridade

class Node:
    def __init__(self, paciente):
        self.paciente = paciente
        self.proximo = None

```

20. Fila de atendimento Chorar:

class FilaAtendimento: ... A fila deverá possuir:

início fim Representação:

início fim ↓ ↓ Ana → Bruno → Carlos → None

```

class FilaAtendimento:
    def __init__(self):

```

```
self.inicio = None
self.fim = None
```

21. Funcionalidades obrigatórias Implemente:

adicionar(paciente) atender() listar() esta_vazia() tamanho()

```
class Paciente:
    def __init__(self, nome, idade, prioridade):
        self.nome = nome
        self.idade = idade
        self.prioridade = prioridade

class Node:
    def __init__(self, paciente):
        self.paciente = paciente
        self.proximo = None

class FilaAtendimento:
    def __init__(self):
        self.inicio = None
        self.fim = None
        self._tamanho = 0

    # 1. Adicionar paciente
    def adicionar(self, paciente):
        novo = Node(paciente)

        if self.esta_vazia():
            self.inicio = novo
            self.fim = novo
        else:
            self.fim.proximo = novo
            self.fim = novo

        self._tamanho += 1

    # 2. Atender o primeiro paciente
    def atender(self):
        if self.esta_vazia():
            return None

        paciente = self.inicio.paciente
        self.inicio = self.inicio.proximo

        if self.inicio is None:
            self.fim = None

        self._tamanho -= 1

        return paciente

    # 3. Listar pacientes
    def listar(self):
        atual = self.inicio

        while atual is not None:
            paciente = atual.paciente

            print(
                paciente.nome,
                "-",
                paciente.idade,
                "anos -",
                paciente.prioridade
            )

            atual = atual.proximo

    # 4. Verificar se a fila está vazia
    def esta_vazia(self):
        return self.inicio is None

    # 5. Informar o tamanho da fila
    def tamanho(self):
        return self._tamanho
```

22. Exemplo de utilização fila = FilaAtendimento()

```

fila.adicionar(Paciente("Ana", 32, "Normal")) fila.adicionar(Paciente("Bruno", 70, "Prioridade")) fila.adicionar(Paciente("Carlos", 45,
"Normal"))

fila.listar()

paciente = fila.atender() print("Atendendo:", paciente.nome)

```

```

fila = FilaAtendimento()

fila.adicionar(Paciente("Ana", 32, "Normal"))
fila.adicionar(Paciente("Bruno", 70, "Prioridade"))
fila.adicionar(Paciente("Carlos", 45, "Normal"))

fila.listar()

paciente = fila.atender()
print("Atendendo:", paciente.nome)

```

23. Desafio — Atendimento prioritário Amplie o sistema.

Pacientes com prioridade devem ser atendidos antes dos pacientes normais.

Exemplo:

Fila inicial:

Ana - Normal Bruno - Normal Carlos - Prioridade Ordem esperada:

1. Carlos
2. Ana
3. Bruno Antes de programar:

descreva a solução; desenhe a estrutura; escolha onde o paciente será inserido; implementar; teste.

```

class Paciente:
    def __init__(self, nome, idade, prioridade):
        self.nome = nome
        self.idade = idade
        self.prioridade = prioridade

class Node:
    def __init__(self, paciente):
        self.paciente = paciente
        self.proximo = None

class FilaAtendimento:
    def __init__(self):
        self.inicio = None
        self.fim = None
        self._tamanho = 0

    def adicionar(self, paciente):
        novo = Node(paciente)

        if self.esta_vazia():
            self.inicio = novo
            self.fim = novo

        elif paciente.prioridade == "Prioridade":
            novo.proximo = self.inicio
            self.inicio = novo

        else:
            self.fim.proximo = novo
            self.fim = novo

        self._tamanho += 1

    def atender(self):
        if self.esta_vazia():
            return None

        paciente = self.inicio.paciente
        self.inicio = self.inicio.proximo

        if self.inicio is None:
            self.fim = None

        self._tamanho -= 1

```

```

        return paciente

    def listar(self):
        atual = self.inicio

        while atual is not None:
            paciente = atual.paciente
            print(
                paciente.nome,
                "-",
                paciente.idade,
                "anos -",
                paciente.prioridade
            )
            atual = atual.proximo

    def esta_vazia(self):
        return self.inicio is None

    def tamanho(self):
        return self._tamanho

# Teste

fila = FilaAtendimento()

fila.adicionar(Paciente("Ana", 32, "Normal"))
fila.adicionar(Paciente("Bruno", 70, "Normal"))
fila.adicionar(Paciente("Carlos", 45, "Prioridade"))

fila.listar()

paciente = fila.atender()
print("Atendendo:", paciente.nome)

```

24. Trabalho colaborativo Forme um grupo de 2 a 4 estudantes.

Etapas 1 — Modelar Paciente → Node → Node → Node Etapas 2 — Implementar Criar:

Paciente Node FilaAtendimento Etapas 3 — Testar Teste:

filas; um paciente; vários pacientes; negro; remoção do último paciente; paciente prioritário. Etapas 4 — Depurar Encontre e corrija pelo menos um erro de proposta.

```

# =====
# TRABALHO COLABORATIVO - FILA DE ATENDIMENTO
# =====

# ETAPA 1 - MODELAR
# Paciente -> Node -> Node -> Node

# ETAPA 2 - IMPLEMENTAR

class Paciente:
    def __init__(self, nome, idade, prioridade):
        self.nome = nome
        self.idade = idade
        self.prioridade = prioridade

class Node:
    def __init__(self, paciente):
        self.paciente = paciente
        self.proximo = None

class FilaAtendimento:
    def __init__(self):
        self.inicio = None
        self.fim = None
        self._tamanho = 0

    def adicionar(self, paciente):
        novo = Node(paciente)

        if self.esta_vazia():
            self.inicio = novo
            self.fim = novo

```

```

        elif paciente.prioridade == "Prioridade":
            novo.proximo = self.inicio
            self.inicio = novo

        else:
            self.fim.proximo = novo
            self.fim = novo

        self._tamanho += 1

    def atender(self):
        if self.esta_vazia():
            return None

        paciente = self.inicio.paciente
        self.inicio = self.inicio.proximo

        if self.inicio is None:
            self.fim = None

        self._tamanho -= 1

        return paciente

    def listar(self):
        atual = self.inicio

        while atual is not None:
            paciente = atual.paciente

            print(
                paciente.nome,
                "-",
                paciente.idade,
                "anos -",
                paciente.prioridade
            )

            atual = atual.proximo

    def esta_vazia(self):
        return self.inicio is None

    def tamanho(self):
        return self._tamanho

# =====
# ETAPA 3 - TESTAR
# =====

fila = FilaAtendimento()

# Teste: fila vazia
print("Fila vazia?", fila.esta_vazia())

# Teste: um paciente
fila.adicionar(Paciente("Ana", 32, "Normal"))

print("\nUm paciente:")
fila.listar()

# Teste: vários pacientes
fila.adicionar(Paciente("Bruno", 70, "Normal"))
fila.adicionar(Paciente("Carlos", 45, "Normal"))

print("\nVários pacientes:")
fila.listar()

# Teste: paciente prioritário
fila.adicionar(Paciente("Daniel", 60, "Prioridade"))

print("\nPaciente prioritário:")
fila.listar()

# Teste: remoção
paciente = fila.atender()

```

```

print("\nPaciente atendido:", paciente.nome)

print("\nFila depois da remoção:")
fila.listar()

# Teste: tamanho
print("\nQuantidade de pacientes:", fila.tamanho())

# Teste: remoção até ficar vazia
while not fila.esta_vazia():
    paciente = fila.atender()
    print("Atendendo:", paciente.nome)

print("\nFila vazia?", fila.esta_vazia())

# =====
# ETAPA 4 - DEPURAR
# =====

# Erro encontrado:
# Quando o último paciente é removido,
# o "fim" também precisa voltar para None.

# Correção feita no método atender():

# if self.inicio is None:
#     self.fim = None

```

25. Exercício de revisão — Sistema de atendimento Desenvolva individualmente uma versão funcional.

Lista de verificação:

Criar Paciente; Criar Node; Criar FilaAtendimento; Implementar adicionar(); Implementar atender(); Implementar listar(); Implementar esta_vazia(); Implementar tamanho(); Testar pelo menos cinco pacientes; Documente um erro encontrado e sua correção.

```

# =====
# EXERCÍCIO DE REVISÃO - SISTEMA DE ATENDIMENTO
# =====

# 1. Criar Paciente
class Paciente:
    def __init__(self, nome, idade, prioridade):
        self.nome = nome
        self.idade = idade
        self.prioridade = prioridade

# 2. Criar Node
class Node:
    def __init__(self, paciente):
        self.paciente = paciente
        self.proximo = None

# 3. Criar FilaAtendimento
class FilaAtendimento:
    def __init__(self):
        self.inicio = None
        self.fim = None
        self._tamanho = 0

# 4. Implementar adicionar()
def adicionar(self, paciente):
    novo = Node(paciente)

    if self.esta_vazia():
        self.inicio = novo
        self.fim = novo

    elif paciente.prioridade == "Prioridade":
        novo.proximo = self.inicio
        self.inicio = novo

    else:

```

```

        self.fim.proximo = novo
        self.fim = novo

    self._tamanho += 1

# 5. Implementar atender()
def atender(self):
    if self.esta_vazia():
        return None

    paciente = self.inicio.paciente
    self.inicio = self.inicio.proximo

    if self.inicio is None:
        self.fim = None

    self._tamanho -= 1

    return paciente

# 6. Implementar listar()
def listar(self):
    atual = self.inicio

    while atual is not None:
        paciente = atual.paciente

        print(
            paciente.nome,
            "-",
            paciente.idade,
            "anos -",
            paciente.prioridade
        )

        atual = atual.proximo

# 7. Implementar esta_vazia()
def esta_vazia(self):
    return self.inicio is None

# 8. Implementar tamanho()
def tamanho(self):
    return self._tamanho

# =====
# 9. TESTE COM PELO MENOS CINCO PACIENTES
# =====

fila = FilaAtendimento()

fila.adicionar(Paciente("Ana", 32, "Normal"))
fila.adicionar(Paciente("Bruno", 70, "Normal"))
fila.adicionar(Paciente("Carlos", 45, "Prioridade"))
fila.adicionar(Paciente("Daniel", 28, "Normal"))
fila.adicionar(Paciente("Elisa", 65, "Prioridade"))

print("=== PACIENTES NA FILA ===")
fila.listar()

print("\nQuantidade de pacientes:", fila.tamanho())

print("Fila vazia?", fila.esta_vazia())

# Atendendo pacientes
print("\n=== ATENDIMENTO ===")

paciente = fila.atender()

if paciente is not None:
    print("Atendendo:", paciente.nome)

print("\n=== FILA APÓS O ATENDIMENTO ===")
fila.listar()

```

```

print("\nQuantidade de pacientes:", fila.tamanho())

# =====
# 10. DOCUMENTAÇÃO DO ERRO E CORREÇÃO
# =====

print("\n== ERRO ENCONTRADO E CORREÇÃO ==")

print("""
Erro encontrado:
Ao remover o último paciente da fila, o atributo 'fim'
poderia continuar apontando para o paciente removido.

Correção:
Depois de atualizar o início da fila, verificamos se ele
ficou igual a None. Se ficou, também colocamos o fim como None.

Código da correção:

if self.inicio is None:
    self.fim = None
""")

```

26. Lista de verificação de aprendizagem Entenda o conceito de endereço de memória. Sei diferenciar ponteiro de referência. Sei utilizar id(). Entenda o compartilhamento de referências. Sei diferença referência e cópia. Entendi objetos mutáveis. Entenda passagem de objetos para funções. Consigo criar um Node. Consigo conectar-nos. Consigo percorrer uma estrutura encadeada. Consigo identificar erros em uma estrutura dinâmica. Consigo implementar uma fila. Consigo explicar minha solução.

```

# =====
# 26. LISTA DE VERIFICAÇÃO DE APRENDIZAGEM
# =====

# ✓ Entendo o conceito de endereço de memória.

# ✓ Sei diferenciar ponteiro de referência.

# ✓ Sei utilizar id().
a = [10, 20, 30]
print(id(a))

# ✓ Entendo o compartilhamento de referências.
a = [10, 20, 30]
b = a

print(a is b) # True

# ✓ Sei a diferença entre referência e cópia.
a = [10, 20, 30]

b = a
c = a.copy()

print(a is b) # True
print(a is c) # False

# ✓ Entendo objetos mutáveis.
a = [10, 20, 30]
a.append(40)

print(a) # [10, 20, 30, 40]

# ✓ Entendo a passagem de objetos para funções.
def adicionar_valor(lista):
    lista.append(50)

numeros = [10, 20, 30]
adicionar_valor(numeros)

print(numeros) # [10, 20, 30, 50]

# ✓ Consigo criar um Node.
class Node:
    def __init__(self, valor):
        self.valor = valor
        self.proximo = None

# ✓ Consigo conectar nós.
n1 = Node("A")

```

```
n2 = Node("B")
n3 = Node("C")

n1.proximo = n2
n2.proximo = n3

# ✓ Consigo percorrer uma estrutura encadeada.
atual = n1

while atual is not None:
    print(atual.valor)
    atual = atual.proximo

# Saída:
# A
# B
# C

# ✓ Consigo identificar erros em uma estrutura dinâmica.
# Exemplo:
# Se esquecer "atual = atual.proximo",
# o while ficará repetindo o mesmo nó.

# ✓ Consigo implementar uma fila.
class Fila:
    def __init__(self):
        self.inicio = None
        self.fim = None

# ✓ Consigo explicar minha solução.
# A fila utiliza nós conectados por referências.
# O início aponta para o primeiro nó e o fim aponta
# para o último nó da estrutura.
```

27. Reflexão final O que é uma referência em Python?

Uma referência em Python é uma variável que aponta para um objeto na memória. Quando duas variáveis fazem referência ao mesmo objeto, elas apontam para o mesmo objeto na memória.

Qual a diferença entre `b = a` e `b = a.copy()`?

`b = a` faz `a` e `b` apontarem para o mesmo objeto, enquanto `b = a.copy()` cria uma cópia independente.

Por que uma estrutura encadeada precisa de referências?

Porque as referências permitem conectar os nós e percorrer a estrutura de um nó para o próximo.

Qual erro você encontrou durante a depuração?

O erro era a falta da linha `atual = atual.proximo`, que fazia o programa entrar em um loop infinito.

Como referências ajudam a compreender estruturas dinâmicas?

As referências permitem que os nós sejam conectados e que a estrutura cresça ou seja modificada dinamicamente, sem precisar criar novos objetos para cada nó.